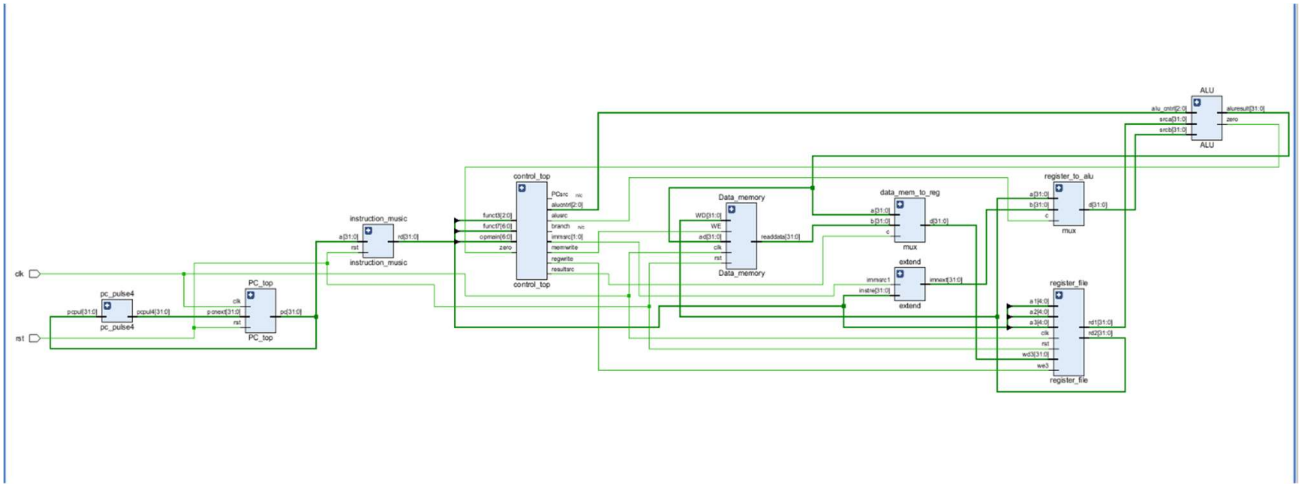
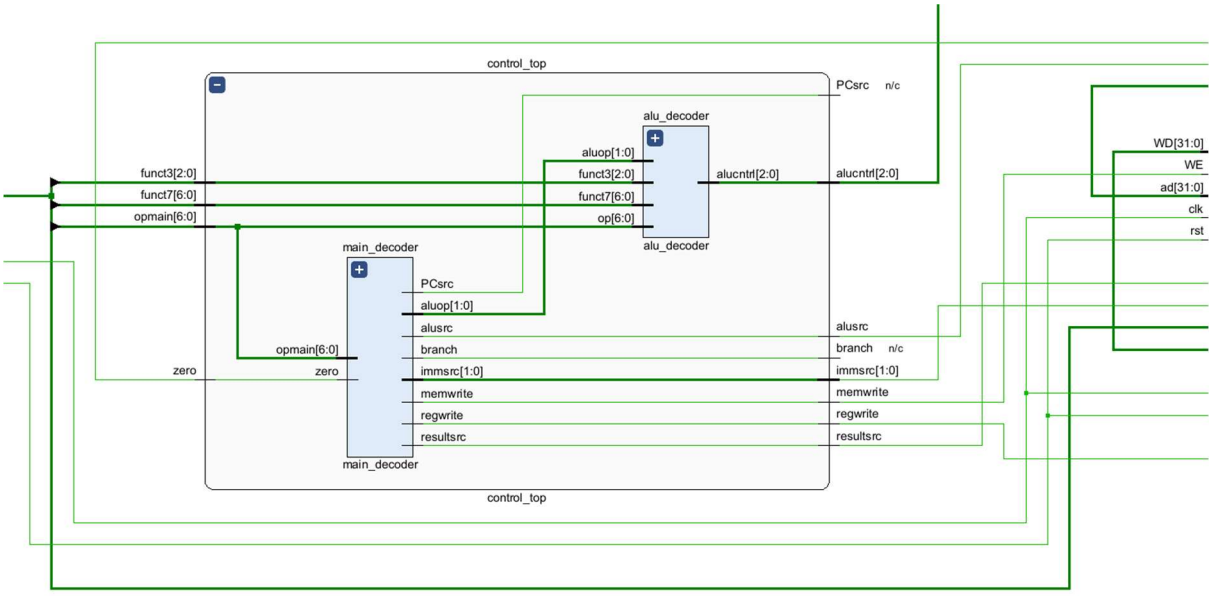
Module Breakdown & RTL Schematics

1. The Top-Level Module



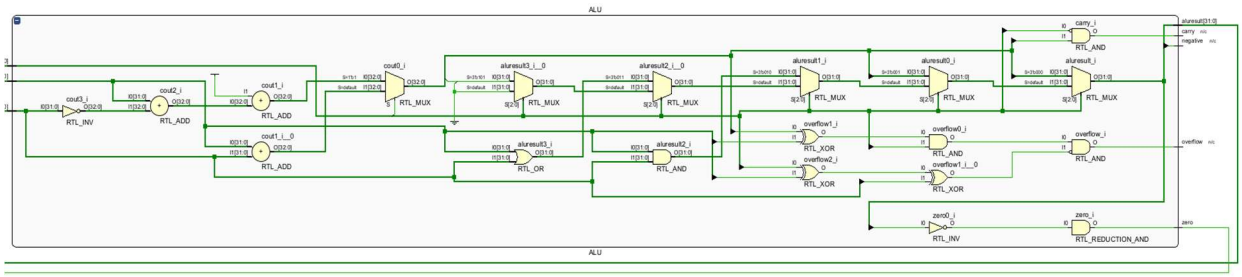
- **Description:** This is the highest level of the hardware design, wiring all the individual sub-modules together to form the complete processor datapath.
- **Key Inputs:** clk (System Clock), rst (Active-low Reset).
- **Key Outputs:** Internal connections routing PC, ALU results, and memory data back to the register file.

2. Control Unit


control_top
PCsrc
n/c
alu_decoder
aluop[1:0]
funct3[2:0]
funct7[6:0]
op[6:0]
aluop[1:0]
funct3[2:0]
funct7[6:0]
opmain[6:0]
aluop[1:0]
aluop[1:0]
aluop[1

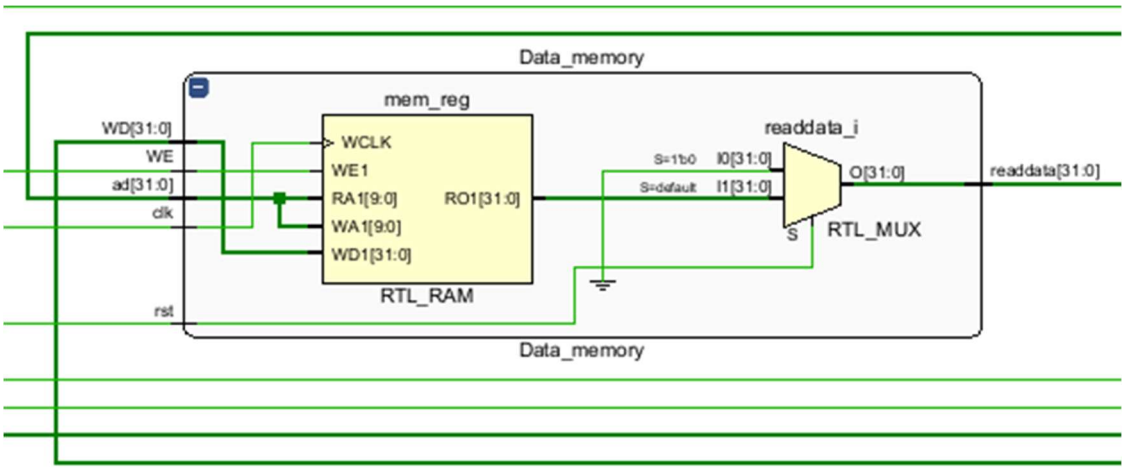
    - funct3[2:0] & funct7[6:0]: Function fields for specific operation decoding.
    - Zero: Flag from the ALU (used for branching).
- **Outputs:**
    - RegWrite, MemWrite: Enable signals for registers and memory.
    - ResultSrc, ALUSrc, ImmSrc: Multiplexer selection signals to route data correctly.
    - alu_cntrl[2:0]: Tells the ALU which specific operation to perform (e.g., ADD, SUB, AND).

3. Arithmetic Logic Unit (ALU)



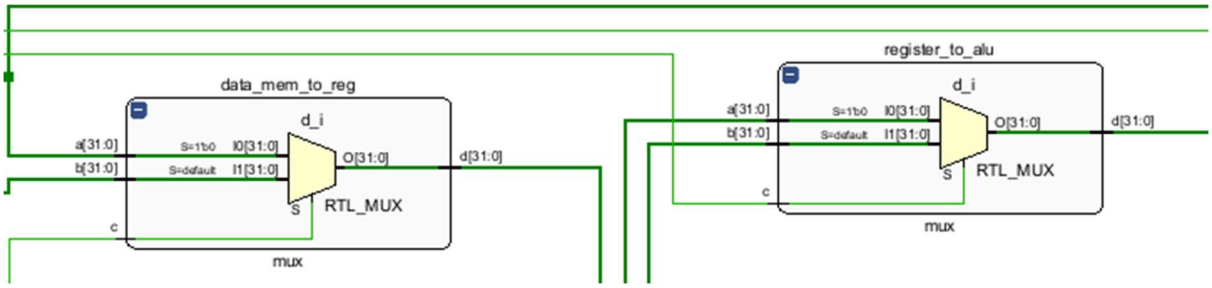
- **Description:** The computational core. The RTL shows a robust network of Adders (RTL_ADD), Inverters (RTL_INV), and Logic Gates (AND, OR, XOR) routed through multiplexers.
- **Inputs:** * srca[31:0], srcb[31:0]: The two 32-bit operands.
    - alu_cntrl[2:0]: The 3-bit control signal dictating the math/logic operation.
- **Outputs:** * aluresult[31:0]: The 32-bit computed answer.
    - Zero: Goes high (1) if the result is 0

4. Data Memory



- **Description:** Handles S-type (Store) and I-type (Load) instructions. Synthesized as an RTL_RAM block.
- **Inputs:** * clk (Clock).
    - WE (Write Enable): High when storing data.
    - ad[31:0] (Address): Where in memory to read/write.
    - WD[31:0] (Write Data): The data to be saved.
- **Outputs:** * readdata[31:0]: The data fetched from memory , an extra mux is used to apply rst feature in it .

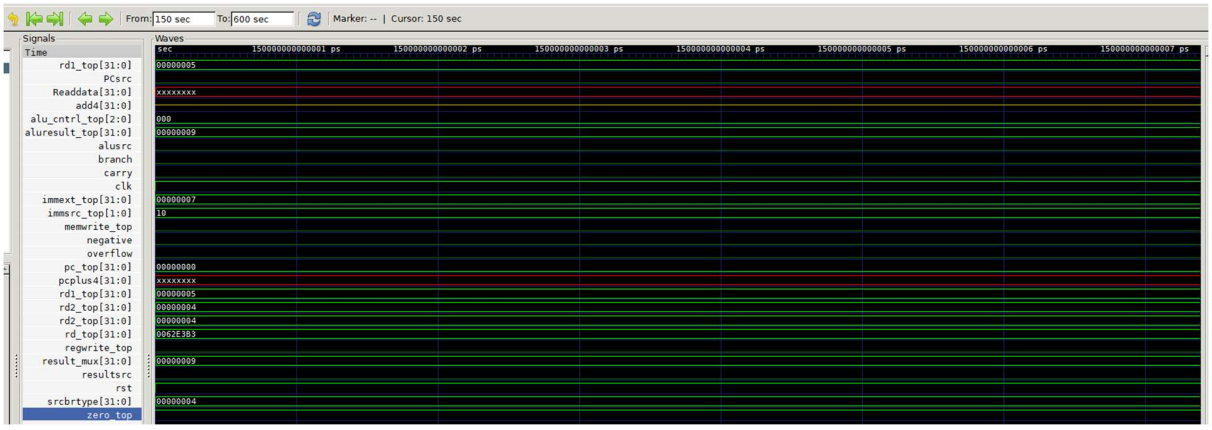
5. Data Routing (Multiplexers)



- **Description:** Muxes control the flow of data based on the Control Unit's decisions.
    - data_mem_to_reg (ResultSrc Mux): Decides if the Register File receives data from the ALU or the Data Memory.

- register_to_alu (ALUSrc Mux): Decides if the ALU's second operand comes from a Register or the Sign-Extended Immediate value.
- These two extra mux are used so that we can add our s – type command in it .
- We use imm_extnd for using immediate type command in it

- **Simulation & Proof of Execution**



To verify the processor's logic, an ADD instruction was simulated to compute **5 + 4**. The waveform demonstrates flawless execution:

- **Inputs Loaded:**
  - rd1_top (Source Register 1) = 00000005 (Hex for 5)
  - rd2_top (Source Register 2) = 00000004 (Hex for 4)
  - These two input in our register_file.v and for this we used

```
    initial begin
        register[0] = 32'h00000000;
        register[1] = 32'h00000001;
        register[2] = 32'h00000002;
        register[3] = 32'h00000003;
        register[4] = 32'h00000004;
        register[5] = 32'h00000005;
        register[6] = 32'h00000004;
    end
```

  - We initialise the s-type command in instruction_music.v that is

```
    initial begin
        mem[0] = 32'h0062E3B3;
    end
```

- **Control Signal:**
  - alu_cntrl_top = 000 (Binary for the ADD operation).
- **Execution & Output:**
  - aluresult_top dynamically updates to 00000009 (Hex for 9).
  - result_mux correctly routes the 00000009 to the write-back stage.

This confirms the datapath accurately fetches, decodes, calculates, and routes standard R-type arithmetic instructions!